Capstone 7-B — Agent Evolution: Order Exception

Build the SAME B2B order exception agent THREE times — first by hand with the raw API, then with the Agent SDK and Claude Code, then from a spec document. Same five operations tools, same mock orders, same business question. Three radically different developer experiences.

A — Healthcare Pre-Auth | B — B2B Order Exception | C — UCC Risk Analyzer

Project Brief

THREE WAYS TO BUILD A HOUSE

Imagine you decide to build the same house three times. The first time, you fell every tree, mill every plank, and hammer every nail by hand. You learn exactly where the load-bearing walls go, why the joists are spaced 16 inches apart, and what happens when a sill plate is undersized. The build takes six months, but you understand every joint in the structure.

The second time, you order pre-cut lumber, pre-fab trusses, and use a nail gun. You finish in six weeks. You build twice as fast and the structure is just as strong, but only because you already know what a structure should look like.

The third time, you hand a contractor a set of architectural drawings. Two weeks later, the house is done — built by people you never met to specifications you wrote in plain English. The previous two builds taught you what to draw and what to leave to the crew.

This capstone is those three builds, compressed into one week. You will write the same B2B order-exception agent three ways. By the end, you will know in your hands — not just in theory — why each layer of abstraction exists, what it costs, and when to reach for each one.

WHAT YOU'LL BUILD

You build a B2B Order Exception Agent. It takes a flagged purchase order — PO-2024-5678 — and walks through five tool calls: pulls order details from the ERP, tracks the shipment with the carrier, checks contract pricing against the invoice, queries available inventory, and drafts a customer notification. The output is a structured exception report (type + root cause + proposed resolution) plus a customer-tier-appropriate email draft.

You build it three times:

- ITERATION 1 Raw API loop — `~250 lines`, `~3 hours`, hand-coded everything (M15B way)
- ITERATION 2 Agent SDK + Claude Code — `~120 lines`, `~2 hours` (M25–M26 way)
- ITERATION 3 Spec-driven — `~100 lines of spec`, `~1 hour` (production way)

SLA & TONE CONSTRAINTS (THE B2B-SPECIFIC COMPLIANCE LAYER)

B2B notifications have to **match customer-tier tone** (terse for enterprise, warmer for SMB), **reference contract clauses where relevant**, and **never overpromise resolution timelines**. In Iteration 1 you encode tone rules in the system prompt and check the output by hand. In Iteration 2 you add an output guardrail hook that runs a tone classifier or pattern check. In Iteration 3 you specify the rules and Claude Code generates both the prompt rules AND the output hook. The pattern is identical to HIPAA in Domain A or PII redaction in Domain C — the spec encodes compliance, the iterations only differ in *where* that compliance lives.

WHY IT MATTERS

Production teams do not pick "raw API vs SDK vs spec" in the abstract — they pick based on what the team already understands and what the problem requires. If you skip Iteration 1, you cannot debug Iteration 3 when the generated code does something weird. If you skip Iteration 3, you are 10x slower than the teams shipping agents in 2026. The point of building the same thing three times is that the *differences* teach you the trade-offs.

Three Iterations at a Glance

Same agent, same business question, same five tools, same mock data. Three layers of abstraction.

	Iter 1 — Raw API	Iter 2 — Agent SDK + Claude Code	Iter 3 — Spec-Driven
LINES YOU WRITE	~250 lines of Python	~120 lines (SDK + hooks + sessions)	~100 lines of spec (no agent code)
TIME TO BUILD	~3 hours	~2 hours	~1 hour
LOOP LIVES IN	Your `while True`	SDK's `query()`	Generated for you by Claude Code
DEBUG METHOD	`print()` in the loop + manual message inspection	Hooks as probes + Anthropic Console + Langfuse traces	Spec ↔ code comparison + tests + evals
KEY INSIGHT	Builds the mental model — every line is yours	Half the code; modular debug; sessions + hooks for free	The spec IS the documentation auditors read

The Three-Iteration Concept

Each iteration produces a working agent that solves the SAME order exception with the SAME five tools and SAME mock data. The agent's *output* is identical across all three. What changes is everything around the agent: the lines you wrote, the time you spent, the abstractions you used, and the way you debug when something breaks.

An agent is "a system prompt + tools + a loop." Iteration 1 makes you build all three from scratch. Iteration 2 keeps the system prompt and tools the same, but the SDK runs the loop for you. Iteration 3 keeps everything the same, but Claude Code writes the prompt, tools, AND loop from a spec you gave it. The agent is unchanged. You changed.

> ***A COMMON MISUNDERSTANDING***
>
> ***"Iteration 3 is just better — why bother with the others?"*** *Because Iteration 3's generated code is not magic. When it produces a `track_shipment` that returns the wrong field name and the agent then mis-reads "delivered" as "in transit," you have to read the generated `tools.py`, find the bug, and fix it — either in the code or in the spec. Without Iteration 1's familiarity with what tool calls and message shapes look like, you cannot tell what the generated code is doing wrong.*

The Scenario — Order Exception Agent

The agent investigates problematic purchase orders: identifies the exception type (delayed shipment, partial delivery, pricing discrepancy, quality hold), gathers data from ERP, WMS, and carrier systems, determines root cause, and proposes resolution with a customer communication draft.

> **BUSINESS QUESTION (USE THIS IN ALL THREE ITERATIONS)**
>
> "PO-2024-5678 from Acme Corp has a flagged exception. Investigate, determine the root cause, and draft a customer notification with a resolution proposal."

TOOLS (5)

- `get_order_details(po_number)` — line items, status, dates, customer tier
- `track_shipment(tracking_number)` — carrier status & location
- `check_contract_pricing(customer_id, sku)` — contract vs invoiced price
- `query_inventory(sku, warehouse)` — stock availability
- `draft_notification(customer_id, type, resolution)` — email draft with tier-aware tone

MOCK DATA SHAPE

- **10 purchase orders** with 3 exception types
- **Exceptions:** delayed shipment (4), partial delivery (3), pricing discrepancy (3)
- **Customers:** Acme Corp (Enterprise), Globex Industries (Enterprise), Initech LLC (SMB), Stark Enterprises (Enterprise), Wayne Industries (SMB)
- **Files:** `orders.json`, `tracking.json`, `contracts.json`, `inventory.json`, `customers.json`

WANT A DIFFERENT DOMAIN?
Switch to Domain A (Healthcare Pre-Auth) or Domain C (UCC Risk Analyzer — default). The lab structure is identical; only the tools and data change.

Architecture Per Iteration

Each iteration produces the same logical agent (system prompt + 5 tools + a loop). What changes is the *physical* architecture — how much you own, how much the SDK owns, and how much is generated for you.

ITER 1 Raw API Loop

You own everything

agent.py *~250 lines, all hand-written*

```
while True:  # the loop you wrote
client.messages.create(...)
if response.stop_reason == "end_turn": break
for block in response.content:
  validate_input · check_cost_cap · log · execute_tool · redact_phi · append · audit_log
```

tools.py	circuit_breaker.py	audit_log.jsonl
5 clinical tools you wrote	*you built*	*you rotate*

server.py (FastAPI) + Dockerfile — you wrote both

ITER 2 Agent SDK + Claude Code

You own the tools and config; SDK owns the loop

agent.py *~90 lines*

```
@tool("lookup_clinical_criteria", ...) ×5 functions
preauth_server = create_sdk_mcp_server(tools=[...])
OPTIONS = ClaudeAgentOptions(system_prompt=..., mcp_servers=..., hooks=...)
async for msg in query(prompt=..., options=OPTIONS): ...
```

The loop, message-passing, retries, and streaming live inside `claude-agent-sdk`. You do not write them.

Claude Code generated	claude-agent-sdk managed
`server.py · Dockerfile · tests · .claude/settings.json · hooks/*.py`	`query() loop · MCP transport · HookMatcher · resume tokens · streaming`

ITER 3 **Spec-Driven** *You own only the spec*

spec/agent-spec.md *~100 lines — the only file you write*

1. Overview
2. Configuration
3. Tools (5)
4. System Prompt
5. Hooks (PHI / tone / risk)
6. Sessions
7. Mock Data
8. API Wrapper
9. Deployment
10. Tests
11. Evaluation
12. File Structure

↓

`/generate-from-spec spec/agent-spec.md`

Claude Code reads the spec, generates ~18 files: `agent.py` · `sessions.py` · `mock_data/*.json` ×4 · `server.py` · `Dockerfile` · `.claude/settings.json` · `hooks/*.py` · `.claude/commands/*.md` · `tests/test_*.py` ×5 · `appendix/manual-loop.py` *(Iter-1 reference).*

Prerequisites

REQUIRED MODULES

- **M05 — Tool Use:** Tool definitions, tool_use blocks, the message loop
- **M12 — ReAct Agents:** Multi-step reasoning across the 5-tool diagnostic flow
- **M15B — Build Complete Agent:** The whole Iter-1 mental model
- **M16–M17 — Guardrails & HITL:** Hooks, especially tone enforcement
- **M19 — Tracing:** Optional but useful for Iter 2 debugging
- **M21, M22B — Deployment:** FastAPI + Docker + Tier 1
- **M25, M26 — Claude Code & Hooks:** CLAUDE.md, slash commands, hooks API

IF YOU HAVE NOT DONE M15B / M26

Iteration 1 is a re-implementation of the M15B reference agent for the order-exception scenario. Do M15B first if you have not built an agent from raw API calls. Iteration 2 leans on the `claude-agent-sdk` patterns taught in M26 (Hooks & Sessions & Agent SDK) — reach for it if `@tool` / `HookMatcher` / `ClaudeAgentOptions` feels unfamiliar.

TOOLS YOU'LL NEED INSTALLED

- Python 3.10+ with pip
- Claude Code CLI (`npm i -g @anthropic-ai/claude-code`) — for Iter 2 and 3
- Docker Desktop for the Tier 1 deployment
- `ANTHROPIC_API_KEY` environment variable
- Optional: a Langfuse account for Iter 2 tracing

SESSION 1

Iteration 1: Raw API Loop

Build the agent the M15B way. You write the loop, the validation, the tone enforcement, the audit, the sessions, and the deployment. Every line is yours. Every bug is yours to find.

`~3 hours` `~250 lines` `7 files` `0 abstractions`

Step 1: Setup & Mock Data

`15 min` `mock_data/*.json`

What & Why: Create the project folder, install `anthropic` + FastAPI, then build the five mock JSON files. Mock data is what separates a "demo" agent from a "doesn't compile" agent — without realistic ERP, carrier, contract, and inventory data, every tool call returns garbage.

SETUP · **MOCK_DATA/**

SETUP

```
mkdir -p agent-iter1-raw/mock_data && cd agent-iter1-raw
python -m venv venv && source venv/bin/activate    # Windows: venv\Scripts\activate
pip install "anthropic≥0.40" "fastapi≥0.110" "uvicorn≥0.27" "pydantic≥2.0"
```

MOCK_DATA/

```
// mock_data/orders.json (excerpt)
{
  "PO-2024-5678": {
    "po_number": "PO-2024-5678",
    "customer_id": "ACME001",
    "submitted": "2024-04-01",
    "promised_ship": "2024-04-08",
    "status": "PARTIALLY_SHIPPED",
    "tracking_numbers": ["1Z999AA10123456784"],
    "line_items": [
      {"sku": "BRG-4892", "qty_ordered": 100, "qty_shipped": 60, "unit_price": 42.00},
      {"sku": "GSK-1175", "qty_ordered": 50,  "qty_shipped": 0,  "unit_price": 18.50}
    ],
    "exception_flagged": "PARTIAL_DELIVERY"
  }
}

// mock_data/customers.json (excerpt)
{
  "ACME001": {"name": "Acme Corporation", "tier": "ENTERPRISE",
              "csm_email": "csm-acme@example.com", "sla_hours": 4},
  "INIT003": {"name": "Initech LLC",       "tier": "SMB",
              "csm_email": "support@example.com", "sla_hours": 24}
}

// mock_data/tracking.json (excerpt)
{
  "1Z999AA10123456784": {
    "carrier": "UPS",
    "status": "DELIVERED",
    "delivered_at": "2024-04-12T14:30:00Z",
    "history": [
      {"ts": "2024-04-09T10:00:00Z", "event": "PICKED_UP", "loc": "Memphis, TN"},
      {"ts": "2024-04-12T14:30:00Z", "event": "DELIVERED", "loc": "Newark, NJ"}
    ]
  }
}

// mock_data/contracts.json (excerpt — for ACME001 + BRG-4892)
{
  "ACME001_BRG-4892": {
    "customer_id": "ACME001", "sku": "BRG-4892",
    "contract_unit_price": 39.50, "volume_tier_min_qty": 100,
    "effective": "2024-01-01", "expires": "2024-12-31"
  }
}

// mock_data/inventory.json (excerpt)
{
  "GSK-1175": {
    "warehouses": {
      "WH-MEMPHIS": {"qty_available": 0,  "qty_reserved": 0},
      "WH-NEWARK":  {"qty_available": 200, "qty_reserved": 50}
    }
  }
}
```

Build all five JSON files with realistic data: 10 POs, 5 customers (3 Enterprise, 2 SMB), tracking for 8 shipments, contracts for 5 SKUs, inventory across 3 warehouses. Cover three exception types: 4 delayed shipments, 3 partial deliveries (incl. PO-2024-5678), 3 pricing discrepancies.

Run: `python -c "import json; print(len(json.load(open('mock_data/orders.json'))), 'POs')"`

```
10 POs
```

Checkpoint

You should see `10 POs`. If you see `0` or a JSON parse error, check that all five files are valid JSON.

Step 2: Define Tools as JSON Schema

`15 min` `tools.py`

What & Why: The Anthropic API needs every tool described as a JSON Schema. PO numbers, SKUs, and tracking numbers all have specific formats (`PO-YYYY-NNNN`, `XXX-NNNN`, UPS/FedEx 12–22 chars). Get the schema right and Claude passes correct types.

TOOLS.PY

TOOLS.PY

```
IMPORT json
FROM pathlib import Path

DATA = Path("mock_data")
ORDERS = json.loads((DATA / "orders.json").read_text())
TRACKING = json.loads((DATA / "tracking.json").read_text())
CONTRACTS = json.loads((DATA / "contracts.json").read_text())
INVENTORY = json.loads((DATA / "inventory.json").read_text())
CUSTOMERS = json.loads((DATA / "customers.json").read_text())

TOOLS = [
    {
        "name": "get_order_details",
        "description": "Get full PO details, status, dates, tracking numbers, customer.",

    # … (full source in the desktop HTML) …

                    else "Let us know how we can help! — Customer Success")
        body = (f"Regarding {args['exception_type']} on your recent order:\n\n"
                f"{args['resolution']}\n\n{sign_off}")
        RETURN {"to": cust.get("csm_email", "ops@example.com"),
                "tier": tier, "salutation": salutation, "body": body}
    RAISE ValueError(f"Unknown tool: {name}")
```

Checkpoint

Run `python -c "from tools import execute_tool; print(execute_tool('get_order_details', {'po_number': 'PO-2024-5678'})['exception_flagged'])"`. Expected: `PARTIAL_DELIVERY`.

Step 3: Build the While Loop

`45 min` `agent.py` `The CORE of Iter 1`

What & Why: The system prompt is critical for order-exception: Claude needs to know to investigate ALL related data (order, tracking, contract, inventory) before drafting a notification, not just react to the first finding.

AGENT.PY (PYTHON) · AGENT.TS (TYPESCRIPT)

AGENT.PY (PYTHON)

```python
IMPORT json, anthropic
FROM tools import TOOLS, execute_tool

client = anthropic.Anthropic()
MODEL = "claude-sonnet-4-6"
SYSTEM = """You are an order-exception specialist. For every flagged PO:
1. get_order_details to understand the situation
2. track_shipment for any tracking numbers (CARRIER VIEW)
3. check_contract_pricing for any line item with potential discrepancy
4. query_inventory for any unshipped or backordered SKUs
5. draft_notification with the right exception_type and a clear resolution

ALWAYS investigate before drafting. Never recommend a credit without checking
contract pricing AND inventory. Match the customer tier's tone (terse for

    # … (full source in the desktop HTML) …

    RETURN text

IF __name__ == "__main__":
    q = "PO-2024-5678 from Acme Corp has a flagged exception. Investigate, " \
        "determine the root cause, and draft a customer notification."
    print(run_agent(q))
```

AGENT.TS (TYPESCRIPT)

```typescript
// agent.ts — the raw API loop. Order-exception investigation.
IMPORT Anthropic from "@anthropic-ai/sdk";
IMPORT { TOOLS, executeTool } from "./tools.js";

const client = NEW Anthropic();
const MODEL = "claude-sonnet-4-6";
const SYSTEM = `You are an order-exception specialist. For every flagged PO:
1. get_order_details to understand the situation
2. track_shipment for any tracking numbers
3. check_contract_pricing for any line item with potential discrepancy
4. query_inventory for any unshipped or backordered SKUs
5. draft_notification with the right exception_type and a clear resolution

ALWAYS investigate before drafting. Never recommend a credit without checking

    # … (full source in the desktop HTML) …

      continue;
    }
    throw NEW Error(`Unexpected stop_reason: ${response.stop_reason}`);
  }
  throw NEW Error(`Agent exceeded ${maxTurns} turns without finishing`);
}
```

Run: `python agent.py`

Expected output (paraphrased — Claude's exact wording varies):

```
Exception Report: PO-2024-5678 (Acme Corp, ENTERPRISE tier)
Type: PARTIAL_DELIVERY
Root cause: 50 units of GSK-1175 unshipped — Memphis warehouse had 0 stock at
pick time. Newark warehouse has 200 units available; can ship within SLA window.

Draft notification (ENTERPRISE tone — formal):
"Team,
Regarding the partial delivery on PO-2024-5678: 60 of 100 units of BRG-4892 have
shipped (UPS tracking 1Z999AA10123456784, delivered Newark NJ 2024-04-12). The
remaining 50 units of GSK-1175 are unshipped due to a Memphis stockout; we have
200 units available in Newark and will dispatch within your 4-hour SLA.
— Operations"
```

✅ **Checkpoint — Step 3**

You should see an exception report identifying PARTIAL delivery, root cause naming the Memphis stockout + Newark availability, and a draft notification with **ENTERPRISE-tier tone** (formal salutation, no "Hi there!"). If the agent drafts an SMB-tone notification ("Hi Acme team!"), the system prompt isn't picking up the customer tier — check that `get_order_details` returns the customer_id and the agent looks it up in the customers data.

Troubleshooting

- **Agent skips track_shipment** → the order has no tracking number yet (e.g., for partial-shipment cases, only the shipped portion has tracking). System prompt should say "call track_shipment for each tracking_number returned by get_order_details — skip if none."
- **Agent recommends a credit instead of investigating** → reinforce the system prompt: "Never recommend a credit without first checking inventory at OTHER warehouses."
- **Draft uses "Hi Acme!" for an ENTERPRISE customer** → the agent didn't read the customer tier. Add explicit guidance: "Look up the customer's tier in customers.json and match tone: ENTERPRISE = formal/terse, SMB = warm/personal."
- **Agent makes up dates or quantities** → system prompt: "Cite ONLY data from tool responses. Never invent numbers."

Step 4: Add Guardrails Manually (incl. Tone Check)

`30 min` `guardrails.py`

What & Why: A loop that does whatever Claude asks is not safe to ship. Production order agents need at minimum: input validation (PO format, SKU format), **output tone enforcement** (enterprise drafts should not start with "Hi there!"), a cost cap, and a circuit breaker. The tone check is the B2B-specific compliance layer.

GUARDRAILS.PY

GUARDRAILS.PY

```
IMPORT re

PO_RE = re.compile(r"^PO-[0-9]{4}-[0-9]{4}$")
SKU_RE = re.compile(r"^[A-Z]{3}-[0-9]{4}$")
COST_LIMIT_TOKENS = 50_000
CIRCUIT_FAIL_THRESHOLD = 3

# Tokens that should NOT appear in ENTERPRISE-tier drafts
ENTERPRISE_BANNED_TOKENS = ["hey there", "hi there", "no worries", "super sorry"]
# Tokens that should NOT appear in any draft (overpromise)
OVERPROMISE_TOKENS = ["guarantee", "definitely tomorrow", "100% certain"]

FUNCTION validate_input(tool_name, args):
    IF tool_name == "get_order_details":

    # … (full source in the desktop HTML) …

CLASS CircuitBreaker:
    FUNCTION __init__(self): self.failures = 0
    FUNCTION record(self, ok):
        self.failures = 0 IF ok else self.failures + 1
        IF self.failures ⩾ CIRCUIT_FAIL_THRESHOLD:
            RAISE RuntimeError("Circuit breaker tripped — aborting")
```

Now wire the guardrails into `_run_messages` in `agent.py`. Replace your existing `_run_messages` with the version below. The tone check fires on `draft_notification` results — if it raises, the error flows back to Claude as a tool error, prompting a redraft.

AGENT.PY (REPLACE _RUN_MESSAGES)

AGENT.PY (REPLACE _RUN_MESSAGES)

```
# Add to imports at the top of agent.py:
FROM guardrails import (validate_input, check_draft_tone,
                        CircuitBreaker, COST_LIMIT_TOKENS)   # NEW

_breaker = CircuitBreaker()        # NEW — module-level instance

FUNCTION _run_messages(messages):
    total_tokens = 0                # NEW — cost cap counter
    FOR turn IN range(MAX_TURNS):
        response = client.messages.create(
            model=MODEL, max_tokens=4096, system=SYSTEM,
            tools=TOOLS, messages=messages,
        )
        total_tokens += (response.usage.input_tokens

    # … (full source in the desktop HTML) …

            messages.append({"role": "user", "content": tool_results})
            CONTINUE

        RAISE RuntimeError(f"Unexpected stop_reason: {response.stop_reason}")

    RAISE RuntimeError(f"Agent exceeded {MAX_TURNS} turns without finishing")
```

Test that the tone check fires on a bad ENTERPRISE draft:

```
# Temporarily edit your draft_notification mock in tools.py to inject bad tone:
#   "Hi there team! No worries about PO-2024-5678 ..."
# Run the agent. The check_draft_tone hook should raise ValueError,
# the error flows back as is_error: true, and Claude redrafts with formal tone.
python agent.py
```

✅ **Checkpoint — Step 4**

The clean PO-2024-5678 query still produces an ENTERPRISE-tier formal draft. With the injected bad tone, the agent receives the tone-check error, redrafts, and the second attempt passes. Restore the clean draft_notification mock when done testing.

TONE ENFORCEMENT IS THE B2B EQUIVALENT OF HIPAA REDACTION

In Domain A you redact PHI; in Domain B you enforce tone. Same hook pattern, different rules. Both are post-tool-use checks that can either reject the result (hard fail) or rewrite it (soft fail). For ENTERPRISE customers, a casual tone in a delay notification can damage the relationship; for SMB, an overly formal tone can feel cold. The tone check is what lets you scale customer comms without manual review.

Troubleshooting

- `ImportError: cannot import name 'check_draft_tone' from 'guardrails'` → `guardrails.py` isn't in the project folder, or the function name is misspelled.
- **Agent never redrafts after tone error** → Claude may interpret the error as terminal. Strengthen the system prompt: "If you receive a tone_violation error from draft_notification, redraft with formal language and try again."
- **Cost cap fires immediately** → `COST_LIMIT_TOKENS = 50_000` is generous; if you mistakenly set it lower, increase it back.

Step 5: Add Audit Logging

`15 min` `audit_log.jsonl`

What & Why: SOX and customer-contract audits require an exception-handling trail. Every tool call needs a timestamped record: case_id (PO number), tool, inputs, output summary, token count.

PYTHON

```python
IMPORT json, datetime
FUNCTION append_audit(tool_name, args, result, tokens, po_number):
    rec = {
        "ts": datetime.datetime.utcnow().isoformat() + "Z",
        "po_number": po_number,
        "tool": tool_name,
        "input": args,
        "output_summary": str(result)[:200],
        "tokens": tokens,
    }
    WITH open("audit_log.jsonl", "a") as f:
        f.write(json.dumps(rec) + "\n")
```

Now wire `append_audit` into `_run_messages`. Add the import and call right after the successful `execute_tool`:

AGENT.PY (ADDITIONS)

```python
# Add to imports at the top of agent.py:
FROM audit import append_audit                    # NEW

# Inside _run_messages, after `result = execute_tool(...)` and the tone check== "draft_notification":
                            check_draft_tone(result)
                        _breaker.record(ok=True)
                        # Extract PO number from the user's first message for audit context
                        po = next(
                            (w FOR w IN messages[0]["content"].split() IF w.startswith("PO-")),
                            "default"
                        )
                        append_audit(                              # NEW
                            tool_name=block.name,
                            args=block.input,
                            result=result,
                            tokens=total_tokens,
                            po_number=po,
                        )
                        tool_results.append({...})
                        # ... rest of try-block unchanged ...
```

Run: `python agent.py`

Then inspect the audit file:

```
cat audit_log.jsonl   # macOS/Linux
type audit_log.jsonl  # Windows cmd
```

Expected output (5 lines, one per tool call, all with the same po_number):


```
{"ts": "2026-05-09T12:01:14Z", "po_number": "PO-2024-5678", "tool": "get_order_details", "input":
{"po_number": "PO-2024-5678"}, "output_summary": "{\"customer_id\": \"ACME001\", \"status\":
\"PARTIALLY_SHIPPED\", ...}", "tokens": 1842}
{"ts": "2026-05-09T12:01:18Z", "po_number": "PO-2024-5678", "tool": "track_shipment", "input":
{"tracking_number": "1Z999AA10123456784"}, "output_summary": "{\"carrier\": \"UPS\", \"status\":
\"DELIVERED\", ...}", "tokens": 2510}
{"ts": "2026-05-09T12:01:22Z", "po_number": "PO-2024-5678", "tool": "query_inventory", "input": {"sku":
"GSK-1175", "warehouse": "WH-NEWARK"}, "output_summary": "{\"qty_available\": 200, ...}", "tokens": 3185}
{"ts": "2026-05-09T12:01:26Z", "po_number": "PO-2024-5678", "tool": "draft_notification", "input":
{"customer_id": "ACME001", "exception_type": "PARTIAL", ...}, "output_summary": "{\"to\": \"csm-
acme@example.com\", \"tier\": \"ENTERPRISE\", ...}", "tokens": 3920}
```


✅ **Checkpoint — Step 5**

You should see 4–5 lines in `audit_log.jsonl` all keyed by `po_number: "PO-2024-5678"`. If `po_number` is "default", the parser didn't find a PO in the user's first message — either the question doesn't include the PO, or the parsing logic doesn't match your prompt format.

Step 6: Multi-Turn PO Sessions

`15 min` `session.py`

What & Why: Ops reviewers ask follow-ups about the same PO: "What if we expedite-ship from Newark instead?" should reuse the prior investigation context. Iter 1 implements this by maintaining a per-PO messages list and reusing the same `_run_messages` helper from Step 3.

Create a new file `session.py` in the project folder:

SESSION.PY

SESSION.PY

```
FROM agent import _run_messages

SESSIONS, list] = {}   # po_number → messages list
WINDOW = 24                     # 5 tools per turn × ~5 turns + buffer

FUNCTION chat(po_number, user_msg):

    msgs = SESSIONS.setdefault(po_number, [])
    msgs.append({"role": "user", "content": user_msg})
    answer, msgs = _run_messages(msgs)
    # Sliding window= msgs[-WINDOW:]
    RETURN answer
```

Try it — multi-turn PO investigation:

```
python -c "
FROM session import chat
print(chat('PO-2024-5678', 'PO-2024-5678 from Acme Corp has a flagged exception. Investigate and draft a
notification.'))
print('---')
print(chat('PO-2024-5678', 'What would change IF we expedite-ship from Newark instead of waiting for Memphis
restock?'))
"
```

Expected behavior: the second call references the prior investigation findings (Memphis stockout, Newark availability) and proposes the expedite scenario as an alternative. Without session continuity, the agent would have no memory of the GSK-1175 issue.

✅ **Checkpoint — Step 6**

The second call references "the GSK-1175 stockout from Memphis" and proposes a different resolution (expedite ship from Newark with revised SLA estimate). If the second call says "I need to investigate first", session isn't carrying over — verify `SESSIONS[po_number]` persists across calls.

Troubleshooting

- `ImportError: cannot import name '_run_messages' from 'agent'` → you skipped Step 3's refactor of `agent.py`. Go back and split the loop into `_run_messages` + `run_agent`.
- **Each call re-investigates** → `SESSIONS` is module-level. If you're calling from separate Python processes (e.g., via subprocess), use a database or Redis instead.

Step 7: Deploy as FastAPI + Docker

20 min | server.py + Dockerfile

SERVER.PY · DOCKERFILE

SERVER.PY

```
FROM fastapi import FastAPI, HTTPException
FROM pydantic import BaseModel
FROM agent import run_agent
FROM session import chat

app = FastAPI()
CLASS Q(BaseModel): question: str
CLASS C(BaseModel): po_number: str; message: str

FUNCTION health(): return {"status": "ok"}

FUNCTION exception_ep(q):
    TRY: return {"report": run_agent(q.question)}
    CATCH: raise HTTPException(500, str(e))

FUNCTION chat_ep(c):
    RETURN {"answer": chat(c.po_number, c.message)}
```

DOCKERFILE

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
ENV PYTHONUNBUFFERED=1
CMD ["uvicorn", "server:app", "--host", "0.0.0.0", "--port", "8000"]
```

Also create `requirements.txt` at the project root:

```
anthropic≥0.40
fastapi≥0.110
uvicorn≥0.27
pydantic≥2.0
```

Run locally first:

```
uvicorn server:app --reload --port 8000
# In another terminal:
curl localhost:8000/health
curl -X POST localhost:8000/exception -H "Content-Type: application/json" \
     -d '{"question":"PO-2024-5678 from Acme Corp has a flagged exception. Investigate."}'
```

Then build and run with Docker:

```
docker build -t iter1-b .
docker run --rm -p 8000:8000 -e ANTHROPIC_API_KEY=$ANTHROPIC_API_KEY iter1-b
```

Iteration 1 Complete — End-to-End Verification

You should now have **11 files** in your project folder:

```
agent-iter1-raw/
|— agent.py            # _run_messages + run_agent
|— tools.py            # 5 ops tool schemas + execute_tool
|— mock_data/
|-   |— orders.json       # 10 POs across 3 exception types
|-   |— tracking.json     # 8 carrier records
|-   |— contracts.json    # 5 customer-SKU contracts
|-   |— inventory.json    # 6 SKUs × 3 warehouses
|-   |— customers.json    # 5 customers (3 ENTERPRISE, 2 SMB)
|— guardrails.py       # validate_input + check_draft_tone + CircuitBreaker
|— audit.py            # SOX-compliant append_audit
|— session.py          # multi-turn PO sessions
```

```
├── server.py           # FastAPI handlers
├── Dockerfile | requirements.txt
```

Run the full Iter-1 acceptance test:

```
# 1. Investigate the canonical PO (Acme, ENTERPRISE)
curl -s -X POST localhost:8000/exception -H "Content-Type: application/json" \
     -d '{"question":"PO-2024-5678 from Acme has a flagged exception. Investigate."}' | python -m json.tool

# 2. Multi-turn PO session (expedite-ship what-if)
curl -s -X POST localhost:8000/chat -H "Content-Type: application/json" \
     -d '{"po_number":"PO-2024-5678","message":"Investigate this exception."}' | python -m json.tool
curl -s -X POST localhost:8000/chat -H "Content-Type: application/json" \
     -d '{"po_number":"PO-2024-5678","message":"What if we expedite-ship from Newark?"}' | python -m json.tool

# 3. Tone enforcement: bad ENTERPRISE draft gets rejected
# (You'd inject "Hi there!" into the draft_notification mock to test this)

# 4. Audit log keyed by PO
docker exec $(docker ps -q --filter ancestor=iter1-b) cat audit_log.jsonl | head -5
# Look for "po_number": "PO-2024-5678" on every line
```

You pass Iter 1 if: (1) /health returns ok; (2) /exception returns a PARTIAL exception report with formal ENTERPRISE-tier draft; (3) the second /chat call references the prior Memphis stockout findings; (4) audit log entries are all keyed by `po_number: "PO-2024-5678"`.

ITERATION 1 METRICS

- **Files:** 11 (agent.py, tools.py, mock_data/×5, guardrails.py, audit.py, session.py, server.py, Dockerfile, requirements.txt)
- **Lines you wrote:** ~250
- **Time:** ~3 hours
- **Abstractions used:** none — just `anthropic` Messages API and FastAPI

Debugging in Iteration 1: Print Statements + Manual Inspection

When the agent gives wrong output in Iter 1, you debug like you debug any Python program: by reading your own code.

A. Add a debug_turn() helper

PYTHON

PYTHON

```
FUNCTION debug_turn(turn_num, response, messages):
    print(f"\n═══ TURN {turn_num} ═══ stop_reason: {response.stop_reason}")
    FOR block IN response.content═ "tool_use":
            print(f"  TOOL: {block.name}({block.input})")
        ELIF block.type ═ "text":
            print(f"  TEXT: {block.text[:120]}...")
    print(f"  Tokens={response.usage.input_tokens} out={response.usage.output_tokens}")
```

B. Inspect messages list manually

The most common Iter-1 bug is malformed messages. Add `print(json.dumps(messages, default=str, indent=2))` before each `messages.create`.

C. Common Iter-1 bugs and how to spot them

- **Wrong `tool_use_id` in tool_result** → 400 from API. Match the `id` on the tool_use block.
- **Forgot to append the assistant turn** → API complains about message order.
- **Domain-specific:** agent drafts a notification BEFORE checking inventory or contract pricing — root cause incomplete. The system prompt did not enforce the investigate-then-draft order. Strengthen with: "ALWAYS call get_order_details, track_shipment, check_contract_pricing, AND query_inventory before draft_notification."
- **Domain-specific:** agent drafts an SMB-tone notification for an ENTERPRISE customer (or vice versa). Either tone-check hook didn't run, or the draft_notification result didn't include the customer tier. Add the tier to the response payload and to the audit log.
- **Domain-specific:** agent recommends a credit when a re-shipment would resolve. Usually means the agent stopped after track_shipment showed DELIVERED without checking inventory for unshipped lines. Re-read the system prompt — PARTIAL delivery requires inventory check.

D. Debug exercise (do this before moving on)

Add a hard tone violation: change `execute_tool` for ENTERPRISE customers so the body starts with "Hey there!". Re-run the agent. The `check_draft_tone` guardrail should raise. Read the error message and trace where the tone got injected. Restore and re-run. **This builds the muscle memory you need to debug Iter 3 generated drafts.**

SESSION 2

Iteration 2: Agent SDK + Claude Code

Now you let the SDK run the loop, hooks handle tone enforcement and validation, sessions handle multi-turn, and Claude Code does most of the typing. Same agent. Half the lines. Different debugging.

`~2 hours` `~120 lines` `8 files` `SDK + hooks + sessions`

Step 8: Create CLAUDE.md via Claude Code

`10 min` `CLAUDE.md`

CLAUDE CODE · CLAUDE.MD

CLAUDE CODE

```
mkdir agent-iter2-sdk && cd agent-iter2-sdk
python -m venv venv && source venv/bin/activate    # Windows: venv\Scripts\activate
pip install "claude-agent-sdk≥0.2" "fastapi≥0.110" "uvicorn≥0.27" "pydantic≥2.0"
npm i -g @anthropic-ai/claude-code   # if not already installed
claude
> /init
```

CLAUDE.MD

```
# Agent: Order Exception Investigation Agent (Iteration 2)

## Stack
- Python 3.11+
- `claude-agent-sdk` (the official Agent SDK — NOT a wrapper around `client.messages.create()`)
- FastAPI + Docker for deployment
- Mock data in mock_data/*.json (5 files, tracking, contracts, inventory, customers)

## File Layout
- agent.py            — query() entry + 5 @tool-decorated MCP tools + create_sdk_mcp_server
- hooks/              — PreToolUse + PostToolUse hook scripts (tone enforcement on draft_notification)
- .claude/settings.json — hooks registration (matchers + commands)
- sessions.py         — multi-PO session resume
- server.py           — FastAPI wrapper

    # … (full source in the desktop HTML) …

2. track_shipment for each tracking_number returned
3. check_contract_pricing for any line with potential discrepancy
4. query_inventory for any unshipped/backordered SKU
5. draft_notification (matching customer tier tone)

NEVER draft before completing the investigation. NEVER overpromise.
```

Step 9: Build Agent with @tool Decorators (claude-agent-sdk)

`15 min` `agent.py`

WHAT THE REAL SDK LOOKS LIKE

If you've used `client.messages.create()` in Iter 1, you might expect the SDK to be a thin `Agent` class wrapping it. It is not. The SDK is built around **MCP tools** + **an async** `query()` **generator** + **options/hooks via** `ClaudeAgentOptions`. Tools return `{"content": [{"type": "text", "text": ...}]}` (MCP shape), not bare Python values.

CLAUDE CODE PROMPT · AGENT.PY (PYTHON) · AGENT.TS (TYPESCRIPT)

CLAUDE CODE PROMPT

```
> Create agent.py using `claude-agent-sdk`. Define five order-ops tools as
> @tool-decorated functions returning MCP-shaped {"content":[...]}:
> get_order_details, track_shipment, check_contract_pricing, query_inventory,
> draft_notification (tier-aware tone). Wire them into a create_sdk_mcp_server,
> build ClaudeAgentOptions with the system prompt from CLAUDE.md, and expose
> run(question) driving query() and concatenating AssistantMessage text.
```

AGENT.PY (PYTHON)

```python
IMPORT json
FROM pathlib import Path
FROM claude_agent_sdk import (
    query, tool, create_sdk_mcp_server,
    ClaudeAgentOptions, AssistantMessage,
)

DATA = Path("mock_data")
ORDERS    = json.loads((DATA / "orders.json").read_text())
TRACKING  = json.loads((DATA / "tracking.json").read_text())
CONTRACTS = json.loads((DATA / "contracts.json").read_text())
INVENTORY = json.loads((DATA / "inventory.json").read_text())
CUSTOMERS = json.loads((DATA / "customers.json").read_text())


    # … (full source in the desktop HTML) …

    FOR msg IN query(prompt=question, options=OPTIONS):
        IF isinstance(msg, AssistantMessage):
            FOR block IN msg.content:
                IF getattr(block, "text", None):
                    parts.append(block.text)
    RETURN "\n".join(parts)
```

AGENT.TS (TYPESCRIPT)

```typescript
// agent.ts — @anthropic-ai/claude-agent-sdk version
IMPORT { query, tool, createSdkMcpServer } from "@anthropic-ai/claude-agent-sdk";
IMPORT { z } from "zod";
IMPORT * as fs from "fs";

const ORDERS    = JSON.parse(fs.readFileSync("mock_data/orders.json", "utf8"));
const TRACKING  = JSON.parse(fs.readFileSync("mock_data/tracking.json", "utf8"));
const CONTRACTS = JSON.parse(fs.readFileSync("mock_data/contracts.json", "utf8"));
const INVENTORY = JSON.parse(fs.readFileSync("mock_data/inventory.json", "utf8"));
const CUSTOMERS = JSON.parse(fs.readFileSync("mock_data/customers.json", "utf8"));

const ok = (p) => ({ content: [{ type, text: JSON.stringify(p) }] });

const getOrderDetails = tool(

    # … (full source in the desktop HTML) …

        IF ("text" in block) parts.push(block.text);
      }
    }
  }
  RETURN parts.join("\n");
}
```

WHAT JUST DISAPPEARED

You no longer write the message loop, the stop_reason check, the tool_result append, or JSON schema dicts. **Same five-tool investigation flow, ~85 lines instead of ~140.**

Troubleshooting

- `ModuleNotFoundError: No module named 'claude_agent_sdk'` → `pip install "claude-agent-sdk ⩾ 0.2"` in your venv.

- `ImportError: cannot import name 'Agent' from 'anthropic'` → you're trying the old fictional API. The real SDK is `claude_agent_sdk`.
- Tool call rejected with "not allowed" → add the tool's `mcp__ops__<name>` entry to `allowed_tools`.

Step 10: Hooks via .claude/settings.json + HookMatcher

`20 min` `hooks/*.py + .claude/settings.json`

What & Why: The SDK supports two hook surfaces: (a) **file-based hooks** in `.claude/settings.json` shelling out to scripts (production-friendly), and (b) **in-process hooks** via `HookMatcher` in `ClaudeAgentOptions(hooks={...})`. We use both: file-based for audit + tone enforcement on `draft_notification`, in-process for input validation that needs to deny.

.CLAUDE/SETTINGS.JSON · HOOKS/TONE_CHECK.PY · IN-PROCESS VALIDATION

.CLAUDE/SETTINGS.JSON

```
{
  "hooks": {
    "PreToolUse": [
      { "matcher": "*", "command": "python hooks/log_call.py" }
    ],
    "PostToolUse": [
      { "matcher": "mcp__ops__draft_notification",
        "command": "python hooks/tone_check.py" },
      { "matcher": "*", "command": "python hooks/audit.py" }
    ]
  }
}
```

HOOKS/TONE_CHECK.PY

```
IMPORT sys, json

ENTERPRISE_BANNED = ["hi there", "hey there", "no worries", "super sorry"]
OVERPROMISE       = ["guarantee", "definitely tomorrow", "100% certain"]

payload = json.load(sys.stdin)
result  = payload.get("tool_result") or {}
# tool_result is wrapped {"content":[{"type":"text","text": json}]}
try= json.loads(result["content"][0]["text"])
CATCH:
    json.dump(payload, sys.stdout); sys.exit(0)

body = (inner.get("body") or "").lower()
tier = inner.get("tier", "SMB")

    # … (full source in the desktop HTML) …

    # Rewrite the tool_result so the agent sees an error and re-drafts.
    err = {"error": "tone_violation", "violations": violations,
           "tier": tier, "advice": "Re-draft with formal/non-overpromising tone."}
    payload["tool_result"] = {"content": [{"type": "text",
                                           "text": json.dumps(err)}]}
json.dump(payload, sys.stdout)
```

IN-PROCESS VALIDATION

```
IMPORT re
FROM claude_agent_sdk import HookMatcher

PO_RE  = re.compile(r"^PO-[0-9]{4}-[0-9]{4}$")
SKU_RE = re.compile(r"^[A-Z]{3}-[0-9]{4}$")

FUNCTION validate_input(input_data, tool_use_id, context):
    name = input_data.get("tool_name", "")
    args = input_data.get("tool_input", {}) or {}
    fail = None
    IF name.endswith("get_order_details") and not PO_RE.match(args.get("po_number", "")):
        fail = f"Invalid PO format: {args.get('po_number')!r}"
    ELIF name.endswith(("check_contract_pricing", "query_inventory")) \
            and not SKU_RE.match(args.get("sku", "")):

    # … (full source in the desktop HTML) …


# Then update OPTIONS in agent.py= ClaudeAgentOptions(
    # ... existing fields ...
    hooks={"PreToolUse": [HookMatcher(matcher="mcp__ops__*",
                                      hooks=[validate_input])]},
)
```

Create the supporting log_call.py and audit.py hooks — same stdin/stdout pattern:

HOOKS/LOG_CALL.PY · HOOKS/AUDIT.PY

HOOKS/LOG_CALL.PY

```
IMPORT sys, json, datetime

payload = json.load(sys.stdin)
ts = datetime.datetime.utcnow().isoformat() + "Z"
print(f"[{ts}] PRE  {payload.get('tool_name','?')}({payload.get('tool_input',{})})",
      file=sys.stderr)
json.dump(payload, sys.stdout)   # pass-through
```

HOOKS/AUDIT.PY

```
IMPORT sys, json, datetime

payload = json.load(sys.stdin)
# Extract PO number from tool_input if present, else 'default'.
ti = payload.get("tool_input", {}) or {}
po = ti.get("po_number") or "default"
rec = {
    "ts": datetime.datetime.utcnow().isoformat() + "Z",
    "po_number": po,
    "tool": payload.get("tool_name"),
    "input": ti,
    "output_summary": str(payload.get("tool_result"))[:200],
}
WITH open("audit_log.jsonl", "a") as f:
    f.write(json.dumps(rec, default=str) + "\n")
json.dump(payload, sys.stdout)   # pass-through
```

Smoke-test each hook standalone:

```
# tone_check should reject ENTERPRISE drafts with banned tokens
echo '{"tool_name":"mcp__ops__draft_notification","tool_result":{"content":[{"type":"text","text":"{\"body\":\"Hi
there team!\",\"tier\":\"ENTERPRISE\"}"}]}}' \
  | python hooks/tone_check.py
# Should rewrite tool_result to an error payload prompting redraft.

# audit should append a record keyed by po_number
echo '{"tool_name":"get_order_details","tool_input":{"po_number":"PO-2024-5678"},"tool_result":"ok"}' \
  | python hooks/audit.py
cat audit_log.jsonl | tail -1
```

Then run the agent end-to-end:

```
python -c "import asyncio, agent; print(asyncio.run(agent.run('Investigate PO-2024-5678 from Acme Corp')))"
```

✅ **Checkpoint — Step 10**

You should see (1) `[timestamp] PRE` log lines on stderr, (2) the agent's exception report on stdout with formal ENTERPRISE-tier draft, (3) `audit_log.jsonl` with entries keyed by PO. If you trigger a tone violation (inject "Hi there!" in your `draft_notification` mock), the tone_check hook rewrites the tool_result as an error and the agent redrafts.

Troubleshooting

- **Tone hook crashes on JSON parse** → the SDK wraps tool results in `{"content":[{"type":"text","text":...}]}`. The `text` field is a JSON STRING that needs `json.loads` to inspect.
- **Agent ignores the rewritten error and uses the bad draft anyway** → check that you assigned the new payload back: `payload["tool_result"] = {"content": [...error payload...]}` before `json.dump`.
- **audit_log.jsonl shows po_number "default"** → only `get_order_details` has a po_number in tool_input. Other tool calls (draft_notification, query_inventory) need extracting po_number from elsewhere; safest to extract from the user's first message in agent.py instead of inside the hook.

Step 11: Sessions — Multi-PO + Fork

`15 min` `sessions.py`

SESSIONS.PY

SESSIONS.PY

```
FROM dataclasses import replace
FROM claude_agent_sdk import query, AssistantMessage
FROM agent import OPTIONS

SESSIONS, str] = {}   # po_number → resume token

FUNCTION _drive(prompt, options):
    parts, sid = [], None
    FOR msg IN query(prompt=prompt, options=options):
        IF isinstance(msg, AssistantMessage):
            FOR block IN msg.content:
                IF getattr(block, "text", None): parts.append(block.text)
        s = getattr(msg, "session_id", None)
        IF s= s

    # … (full source in the desktop HTML) …

FUNCTION what_if(po_number, hypothetical):

    resume = SESSIONS.get(po_number)
    options = replace(OPTIONS, resume=resume) IF resume else OPTIONS
    text, _ = _drive(hypothetical, options)
    RETURN text
```

Try it — multi-PO + fork demo:

```
python -c "
IMPORT asyncio
FROM sessions import chat, what_if

FUNCTION main():
    print('T1, chat('PO-2024-5678', 'PO-2024-5678 from Acme has a flagged exception. Investigate.'))
    print('FORK, what_if('PO-2024-5678', 'What IF we expedite-ship from Newark instead of waiting for Memphis?'))
    print('T2, chat('PO-2024-5678', 'Stick with the original plan. Draft the customer notification.'))

asyncio.run(main())
"
```

✅ **Checkpoint — Step 11**

T1 produces a PARTIAL exception report with Memphis stockout root cause. FORK explores the Newark expedite scenario WITHOUT polluting the main session. T2 references the original plan (not the Newark hypothetical) and produces the formal ENTERPRISE-tier draft. If T2 references Newark, your `what_if` is leaking into `SESSIONS`.

Step 12: Slash Commands

`15 min` `.claude/commands/*.md (3 files)`

What & Why: `/run-exception PO-2024-5678`, `/test-agent`, `/eval-agent` turn the agent into a one-line workflow. Ops reviewers can investigate exceptions without leaving Claude Code.

Create 3 files in `.claude/commands/`:

RUN-EXCEPTION.MD · TEST-AGENT.MD · EVAL-AGENT.MD

RUN-EXCEPTION.MD

```
---
description: Investigate an order exception by PO number
argument-hint: [po_number]
---
Look up $ARGUMENTS in mock_data/orders.json. Build the question string.
Run `python -c "import asyncio, agent; print(asyncio.run(agent.run(q)))"`
where q is the question. Print the exception report, draft, total tokens,
total cost from the ResultMessage emitted by query().
```

TEST-AGENT.MD

```
---
description: Run the unit test suite for the order-exception agent
---
Run `pytest tests/ -v`. Critical tests: test_partial_delivery_resolution
(PO-2024-5678 must identify Memphis stockout + Newark availability),
test_tone_check_rejects_bad_enterprise_draft (tone hook works),
test_audit_keyed_by_po (every audit line has the right po_number).
```

EVAL-AGENT.MD

```
---
description: Run the 10-PO evaluation suite
---
Read test_scenarios.json (10 POs: 4 DELAYED, 3 PARTIAL, 3 PRICING). For each,
call agent.run(), score on: correct exception_type, root cause cited, tier-
appropriate tone, no overpromise. Report per-case score and overall percentage.
```

✅ **Checkpoint — Step 12**

When you type `/` in Claude Code, the three commands appear in autocomplete. `/run-exception PO-2024-5678` investigates and drafts the formal ENTERPRISE notification. `/eval-agent` requires a `test_scenarios.json` file — in Iter 3 the spec generates this for you.

Step 13: Deploy via Claude Code

`15 min` `server.py + Dockerfile`

What & Why: Same FastAPI + Docker pattern as Iter 1, but Claude Code writes it.

CLAUDE CODE PROMPT · RESULTING SERVER.PY · DOCKERFILE

CLAUDE CODE PROMPT

```
> Create server.py and Dockerfile. Endpoints: GET /health,
> POST /exception (single-shot), POST /chat (po_number + message).
> Async FastAPI handlers awaiting agent.run() and sessions.chat() (both
> async coroutines from claude-agent-sdk). python:3.11-slim base, install
> claude-agent-sdk + dependencies, expose 8000. Mount .claude/ into the
> container so settings.json + hook scripts resolve at runtime.
```

RESULTING SERVER.PY

```
FROM fastapi import FastAPI, HTTPException
FROM pydantic import BaseModel
FROM agent import run as agent_run
FROM sessions import chat as session_chat

app = FastAPI(title="Order Exception Agent (Iter 2 — SDK)")

CLASS Q(BaseModel): question: str
CLASS C(BaseModel): po_number: str; message: str

FUNCTION health(): return {"status": "ok", "iter": 2}

FUNCTION exception_ep(q):
    TRY: return {"report": agent_run(q.question)}
    CATCH: raise HTTPException(500, str(e))

FUNCTION chat_ep(c):
    TRY: return {"answer": session_chat(c.po_number, c.message)}
    CATCH: raise HTTPException(500, str(e))
```

DOCKERFILE

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
ENV PYTHONUNBUFFERED=1
CMD ["uvicorn", "server:app", "--host", "0.0.0.0", "--port", "8000"]
```

Run locally first, then Docker:

```
uvicorn server:app --reload --port 8000
# In another terminal:
curl localhost:8000/health
curl -X POST localhost:8000/exception -H "Content-Type: application/json" \
     -d '{"question":"Investigate PO-2024-5678"}'

# Then build the container:
docker build -t iter2-b .
docker run --rm -p 8000:8000 -e ANTHROPIC_API_KEY=$ANTHROPIC_API_KEY iter2-b
```

🎉 Iteration 2 Complete — End-to-End Verification

You should now have **~14 files** in your project:

```
agent-iter2-sdk/
├── CLAUDE.md
├── agent.py            # query() + 5 @tool functions + create_sdk_mcp_server
├── sessions.py
├── mock_data/ ×5       # orders, tracking, contracts, inventory, customers
├── .claude/
│   ├── settings.json
│   ├── commands/run-exception.md, test-agent.md, eval-agent.md
├── hooks/
│   ├── log_call.py, tone_check.py, audit.py
├── server.py | Dockerfile | requirements.txt
```

Acceptance test — same shape as Iter 1:

```
curl -s -X POST localhost:8000/exception -H "Content-Type: application/json" \
     -d '{"question":"Investigate PO-2024-5678"}' | python -m json.tool

curl -s -X POST localhost:8000/chat -H "Content-Type: application/json" \
     -d '{"po_number":"PO-2024-5678","message":"Investigate."}' | python -m json.tool
curl -s -X POST localhost:8000/chat -H "Content-Type: application/json" \
     -d '{"po_number":"PO-2024-5678","message":"What if we expedite from Newark?"}' | python -m json.tool

# Tone enforcement: check audit log shows ENTERPRISE drafts only
docker exec $(docker ps -q --filter ancestor=iter2-b) cat audit_log.jsonl | grep ENTERPRISE
```

You pass Iter 2 if: all 3 outputs are functionally equivalent to Iter 1 but with ~120 lines of code instead of ~250.

Iteration 2 Complete

Same curl as Iter 1; same exception report and tier-appropriate draft.

ITERATION 2 METRICS

- **Files:** ~10 (CLAUDE.md, agent.py, sessions.py, server.py, Dockerfile, .claude/settings.json, hooks/{log_call,tone_check,audit}.py, slash commands) + 5 mock JSON
- **Lines you wrote:** ~120
- **Time:** ~2 hours
- **Abstractions used:** `claude-agent-sdk` (query / @tool / MCP server / ClaudeAgentOptions / HookMatcher) + `.claude/settings.json` + Claude Code

Debugging in Iteration 2: Hooks + Console Web UI + Langfuse

The SDK abstracts the loop — you cannot drop a print inside it. You debug from the outside.

A. Hooks as Debug Probes

Swap `.claude/settings.json` to a debug variant for diagnostic runs (the script writes to stderr and passes the original payload through stdout):

HOOKS/DEBUG.PY · .CLAUDE/SETTINGS.DEBUG.JSON

HOOKS/DEBUG.PY

```
IMPORT sys, json
payload = json.load(sys.stdin)
print(f"[HOOK {payload.get('hook_event_name','?')}] "
      f"{payload.get('tool_name','?')}: "
      f"{json.dumps(payload.get('tool_input', payload.get('tool_result')))[:200]}",
      file=sys.stderr)
json.dump(payload, sys.stdout)
```

.CLAUDE/SETTINGS.DEBUG.JSON

```
{
  "hooks": {
    "PreToolUse":  [{ "matcher": "*", "command": "python hooks/debug.py" }],
    "PostToolUse": [{ "matcher": "*", "command": "python hooks/debug.py" }]
  }
}
```

BETTER than Iter-1 print statements because hooks are modular — symlink `.claude/settings.json` to the debug variant for one run, then back to production.

B. Anthropic Console Web UI

console.anthropic.com shows every API call. **Worked example:** Agent recommends a credit but inventory had stock. Console → Logs → failed call → tool_use shows the agent never called `query_inventory`. Fix: strengthen system prompt to require ALL 4 investigation tools before `draft_notification`. Re-run; ~3 min vs ~15 in Iter 1.

C. Langfuse Traces (if instrumented in M19)

Each PO becomes a waterfall trace: 5 tool calls, each a span. Compare a 2-second simple delay to a 12-second pricing-discrepancy investigation and see exactly which tool took the time (usually `check_contract_pricing` when contracts have many tier rules).

D. /run-exception --verbose

Verbose mode turns on debug hooks for one run. Shows every tool call, hook fire, token count, total cost.

SESSION 3

Iteration 3: Spec-Driven

You stop writing code. You write a spec describing what the order-exception agent should do, then ask Claude Code to build it. ~18 files appear. The spec IS the documentation ops leadership can read.

`~1 hour` `~100 lines of spec` `~18 generated files` `0 lines of agent code`

Step 14: Write agent-spec.md (12 sections)

`30 min` `agent-spec.md`

AGENT-SPEC.MD (EXCERPT)

AGENT-SPEC.MD (EXCERPT)

```
# Agent Specification: Order Exception Investigation Agent

## 1. Overview
Investigates flagged POs, identifies exception type, gathers evidence from
ERP/carrier/contract/inventory systems, drafts tier-appropriate customer
notification with proposed resolution.

## 2. Configuration
- Model: claude-sonnet-4-6
- Framework: claude-agent-sdk (Python). Tools registered via
  create_sdk_mcp_server. Driver: query() with ClaudeAgentOptions.
- Max turns, Max tokens: 4096

## 3. Tools (registered as MCP server "ops", call in this order)

    # … (full source in the desktop HTML) …

|— sessions.py                # resume-token sessions
|— mock_data/ {orders, tracking, contracts, inventory, customers}.json
|— hooks/{log_call,tone_check,audit}.py
|— server.py | Dockerfile | docker-compose.yml | requirements.txt
|— tests/ x5
|— appendix/manual-loop.py     # Iter-1 reference, not for production
```

Step 15: One Command Generates Everything

`10 min` `~18 files appear`

CLAUDE CODE PROMPT

CLAUDE CODE PROMPT

```
> /generate-from-spec spec/agent-spec.md

# (or, if /generate-from-spec is not installed:)
> Read spec/agent-spec.md and build the entire project. Create every file
> in section 12. Use the `claude-agent-sdk` package (NOT a fictional
> anthropic.Agent class). Tools must be @tool-decorated async functions
> registered via create_sdk_mcp_server. Hooks split between
> .claude/settings.json (file-based log/tone/audit) and OPTIONS.hooks
> (HookMatcher in-process validation) per section 5. Generate realistic
> mock B2B data with 10 POs covering all 3 exception types. Also generate
> appendix/manual-loop.py as an under-the-hood reference.
```

What you should see Claude Code create:

```
Reading spec/agent-spec.md ...
Created agent.py (98 lines — 5 @tool functions + create_sdk_mcp_server + ClaudeAgentOptions)
Created sessions.py (32 lines)
Created hooks/log_call.py (12 lines)
Created hooks/tone_check.py (28 lines — banned tokens + overpromise check)
Created hooks/audit.py (18 lines)
Created mock_data/orders.json (10 POs across 3 exception types)
Created mock_data/tracking.json (8 carrier records)
Created mock_data/contracts.json (5 customer-SKU contracts)
Created mock_data/inventory.json (6 SKUs × 3 warehouses)
```

```
Created mock_data/customers.json (5 customers: 3 ENTERPRISE, 2 SMB)
Created server.py (24 lines — async FastAPI)
Created Dockerfile (8 lines)
Created docker-compose.yml (10 lines)
Created requirements.txt (4 lines)
Created CLAUDE.md (38 lines)
Created .claude/settings.json (15 lines)
Created .claude/commands/run-exception.md, test-agent.md, eval-agent.md
Created tests/test_tools.py, test_agent.py, test_hooks.py, test_sessions.py, test_api.py
Created spec/test_scenarios.json (10 scenarios)
Created appendix/manual-loop.py (~80 lines)
Total: 24 files, ~545 generated lines + 100 lines of spec you wrote.
```

Verify it actually works:

```
pytest tests/ -v
```

Expected pytest output (focus on the B2B-critical tests):

```
tests/test_tools.py::test_get_order_details_returns_acme_partial PASSED
tests/test_tools.py::test_query_inventory_newark_has_gsk_1175 PASSED
tests/test_agent.py::test_partial_delivery_resolution_cites_newark PASSED
tests/test_agent.py::test_enterprise_draft_uses_formal_tone PASSED
tests/test_hooks.py::test_tone_check_rejects_bad_enterprise_draft PASSED      # ⟵ B2B-critical
tests/test_hooks.py::test_overpromise_tokens_rejected PASSED                  # ⟵ B2B-critical
tests/test_hooks.py::test_invalid_po_format_denied_by_validator PASSED
tests/test_sessions.py::test_chat_persists_po_context PASSED
tests/test_sessions.py::test_what_if_does_not_pollute_main_po PASSED
tests/test_api.py::test_health_returns_ok PASSED
============================ 10 passed in 13.56s ============================
```

✅ **Checkpoint — Step 15**

All 10 tests pass on the first run. If `test_tone_check_rejects_bad_enterprise_draft` fails, the generated tone_check hook probably checked tone case-sensitively (so "Hi There" passes when "hi there" is banned). Fix the spec section 5 to clarify "case-insensitive substring match" and regenerate.

Troubleshooting

- **Generated tone hook is case-sensitive** → spec ambiguity. Section 5 must say: "compare body.lower() against the banned tokens list."
- **Generated mock_data/customers.json doesn't have ENTERPRISE/SMB tier field** → section 7 of spec should give a sample record per file.
- **tone_check rewrites the draft as a 200-line error** → section 5 should constrain the error format: "Return error payload as JSON object with keys violations, tier, advice."

Step 16: Review & Iterate on the Spec

`15 min` `spec edits + targeted regen`

Example: add a sixth tool `get_carrier_alternatives(origin, destination, sku, qty)` that suggests faster carriers when the current one is delayed. Edit spec section 3 + 11 and ask Claude Code: *"I added get_carrier_alternatives. Update tools.py, mock_data, add a test, and an eval scenario."*

Step 17: Deploy & Compare

`15 min` `same curl, same output`

What & Why: Build, run, curl, compare against Iter 1 and Iter 2.

SHELL

SHELL

```
docker compose up --build -d
docker compose ps   # confirm "Up"

curl localhost:8000/health
# Expected: {"status":"ok","iter":3}

curl -X POST localhost:8000/exception \
     -H "Content-Type: application/json" \
     -d '{"question":"Investigate PO-2024-5678"}' | python -m json.tool
```

Cross-iteration diff — the punchline:

```
# Same query against all three deployments (assumes :8001/8002/8003)
for port in 8001 8002 8003; do
  echo "═══ Iter on :$port ═══"
  curl -s -X POST localhost:$port/exception -H "Content-Type: application/json" \
       -d '{"question":"Investigate PO-2024-5678"}' \
    | python -c "import json, sys; d = json.load(sys.stdin); print((d.get('report') or d.get('answer'))[:300])"
  echo
done
```

Expected: all three return the PARTIAL exception report with Memphis stockout root cause, Newark availability, and a formal ENTERPRISE-tier draft. Wording varies; facts match.

🥕 Iteration 3 Complete — The Whole Capstone

You have **~24 generated files** + your **~100-line spec**:

```
order-exception-agent/
├── spec/agent-spec.md          # YOU wrote this
├── spec/test_scenarios.json    # generated
├── CLAUDE.md | .claude/settings.json | .claude/commands/ ×3
├── agent.py | sessions.py      # generated (SDK)
├── hooks/log_call.py | hooks/tone_check.py | hooks/audit.py
├── mock_data/ ×5               # 10 POs, 8 tracking, 5 contracts, 6 SKUs, 5 customers
├── tests/ ×5                    # generated
├── server.py | Dockerfile | docker-compose.yml | requirements.txt
└── appendix/manual-loop.py     # under-the-hood reference
```

Acceptance test — same shape as Iter 1 and Iter 2:

```
# 1. All 10 generated tests pass
pytest tests/ -v

# 2. End-to-end exception investigation
curl -s -X POST localhost:8000/exception -H "Content-Type: application/json" \
     -d '{"question":"Investigate PO-2024-5678"}' | python -m json.tool

# 3. Multi-PO session via SDK resume tokens
curl -s -X POST localhost:8000/chat -H "Content-Type: application/json" \
     -d '{"po_number":"PO-2024-5678","message":"Investigate."}' | python -m json.tool
curl -s -X POST localhost:8000/chat -H "Content-Type: application/json" \
     -d '{"po_number":"PO-2024-5678","message":"What if we expedite from Newark?"}' | python -m json.tool

# 4. Spec-vs-code drift check (B2B-specific)
claude
> Read spec/agent-spec.md sections 4 and 5. Compare to agent.py and hooks/.
> Check that ENTERPRISE banned tokens are checked case-insensitively.
```

You pass Iter 3 if: (1) all 10 tests pass; (2) PO-2024-5678 returns the PARTIAL report; (3) the second /chat call references prior context; (4) drift check returns "no drift".

ITERATION 3 METRICS

- **Files:** ~24 (all generated by Claude Code from your spec)
- **Lines you wrote:** ~100 (the spec only)
- **Lines Claude Code wrote:** ~545 (readable, runnable, all tests pass)
- **Time:** ~1 hour
- **Abstractions used:** spec format + `/generate-from-spec` + same SDK runtime as Iter 2

Debugging in Iteration 3: Spec Comparison + Tests + Evals

You debug at the spec level. The code is regenerable; the spec is not.

A. Spec vs Code Comparison

CLAUDE CODE PROMPT

CLAUDE CODE PROMPT

```
> Read agent-spec.md and compare it to the generated agent.py and hooks.py.
> Report any deviations — especially around tone enforcement (section 5).
```

B. Test-Driven Debugging

CLAUDE CODE PROMPT

CLAUDE CODE PROMPT

```
> Test test_partial_delivery_resolution failed: expected resolution to mention
> Newark warehouse, got generic "we apologize" text. Read agent-spec.md
> section 4 (System Prompt). Read generated agent.py. Find why the agent
> skipped query_inventory and fix it.
```

C. Eval-Driven Debugging

Run `/eval-agent`. Output: PO-2024-5683 scored 2/4 — "draft tone too casual for ENTERPRISE customer." Fix: strengthen spec section 5 with the explicit banned-token list. Regenerate `hooks.py`. Re-eval → 4/4.

D. Console + Langfuse (same as Iter 2)

THE DEBUGGING EVOLUTION SUMMARY

ITERATION	PRIMARY DEBUG METHOD	SECONDARY	SPEED TO FIX
1 Raw	print() in the loop	Manual message inspection	Slow (find the line)
2 SDK	Hooks + Console Web UI	Langfuse traces	Medium (modular probes)
3 Spec	Spec vs code comparison	Tests + evals + Console	Fast (Claude Code finds it)

The Comparison Table

Same order-exception agent, same business question. Here is everything that changed:

METRIC	ITER 1: RAW API	ITER 2: AGENT SDK	ITER 3: SPEC-DRIVEN
Lines YOU wrote	~250	~120	~100 (spec only)
Time to build	~3 hours	~2 hours	~1 hour
Agent output	Baseline	Same	Same
Tone enforcement	Inline check after draft	One `.claude/settings.json` entry + a 25-line stdin/stdout script	5 lines in spec section 5
Multi-PO sessions	Manual history dict	SDK sessions	SDK sessions (generated)
Adding alternative-carrier tool	Edit 3 files + add validation	One Claude Code prompt	Update spec, ask to regen
Tests (incl. tone)	Manual	Claude Code generated	Spec generates them
Documentation for ops leadership	Separate	CLAUDE.md	Spec IS the doc ops can read
Control over internals	Full	SDK-managed	Least direct (but reviewable)
Understanding needed	Every line	SDK abstractions	Architecture-level
Debugging	print() in loop	Hooks + Console + Langfuse	Spec compare + tests + evals
Onboarding a new ops engineer	Read 7 files	Read CLAUDE.md + 8 files	Read 1 spec file

KEY TAKEAWAY

All three produce the same exception report and tier-appropriate draft. The difference is what each iteration teaches you. Iteration 1 teaches WHAT an agent is. Iteration 2 teaches HOW to build efficiently. Iteration 3 teaches HOW production teams work — *especially* for B2B where the spec doubles as the contract-aware playbook ops can review.